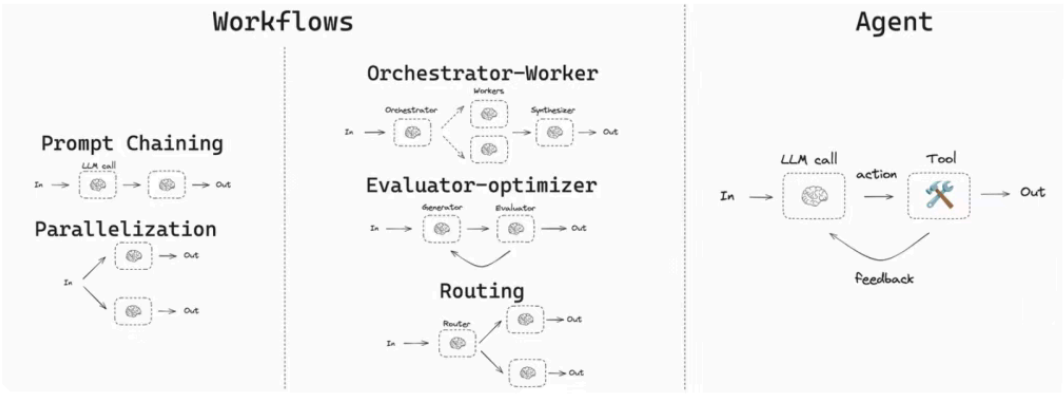
一文讲懂Agent及其主流框架：自己想、自己干、自己复盘的才是好Agent！

原创 腾讯程序员 腾讯技术工程 2025年10月20日 02:36 广东 6098人



> 「Agent不稀奇，能“自己想、自己干、自己复盘”的才是好Agent」可一到落地，名词、框架和坑一起涌来：设计模式、强自治、可控流程、多代理协作.... 到底该不该用Agent？该选哪一类框架？需要用到什么程度？这篇文章用直观的图表、清晰的示例，为你讲清什么是Agent、什么场景适合使用Agent以及各类主流Agent框架，希望能帮各位少走弯路，迅速判断技术路径。

1.Workflow和Agent的区别



在预定义的链路中加入LLM　　在预定义的链路中由LLM控制流程　　LLM根据环境自行选取工具并执行

2.Agent框架选择

核心依赖Github上Star数以及市场热度，综合选取5款Agent框架：

1.**AutoGPT**：Github 17.8w Star

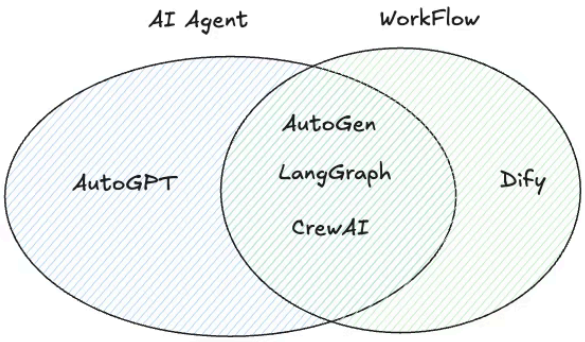
2.**LangGraph**：Github 13.1w Star

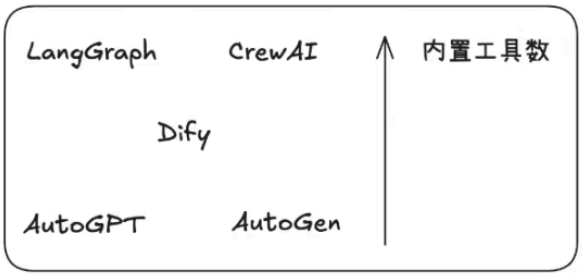
3.**Dify**：Github 11.2w Star

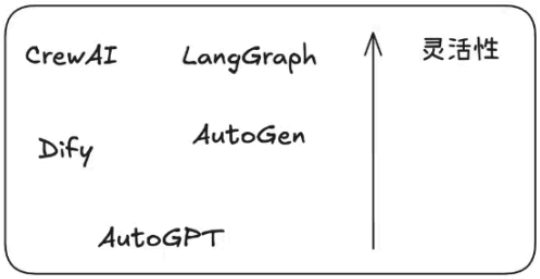
4.**CrewAI**：Github 3w Star

5.**AutoGen**：微软开源 Github 5w Star

3.各Agent框架对比结论







Agent框架	适合场景	优势	不足
AutoGPT	各类通用任务	1.完全自主执行2.任务分解与多步执行3.记忆和持续学习	1.复杂任务场景前后文一致性问题2.高成本和效率问题3.操作可控性较低
LangGraph	可明确拆解任务步骤	1.灵活的多步骤控制2.原生支持短长期记忆3.易调试和全链路可观测	1.自主性有限2.Agent模式不成熟
Dify	可明确拆解任务步骤	1.低代码，易用性与低门槛2.强大的模型与工具能力	1.功能广而不精2.需在简单和复杂场景之间找到平衡
CrewAI	任务步骤不固定，需让Agent自己探索	1.工具和生态集成2.灵活性与深度定制	1.特定功能支持有限（如代码沙盒）
AutoGen	1.原生多代理支持2.灵活的对话流程控制3.可观察调试支持	1.社区生态尚处于起步阶段	

4.为什么需要使用Agent框架

结论：只要“问题不可完全穷举、要跨多系统查证、并且需要在对话中澄清/协商/决策”，就更应该用 Agent 框架，而不是纯 Workflow。

为什么？用一个真实的ToC场景客服链路来说明。

4.1纯 Workflow 在智能客服里的“天花板”

Workflow（无论是 Dify 的可视化编排，还是 LangGraph 的状态机）非常适合步骤确定 + 条件有限的流程，比如：

1.查询订单 → 格式化答复

2.退货→生成标签→发通知

3.FAQ 检索→返回片段

一旦进入长尾问题，Workflow 就会遇到“分支爆炸”：

例：同一条“包裹没到”诉求，可能要综合 ①承运商状态 ②发货 SLA ③节假日政策 ④地址异常 ⑤是否会员 ⑥是否已报缺货 ⑦是否已部分签收 ⑧是否叠加优惠券/补发 等。

如果你用固定分支描述：

假设有 5 个意图 × 6 种物流状态 × 3 种用户等级 × 3 个政策时段（平日/大促/假期） × 3 种地理区域，共5×6×3×3×3=810 条潜在路径。

这还没算异常（报损、拒收、欺诈信号）与“对话澄清”的分支。维护成本和上线速度都会被拖垮。此外，Workflow 对 **对话中的“澄清—再决策—再行动** 并不天然友好，需要把每一步提问、回答、重试都画成节点，复杂而脆弱。

4.2Agent 框架解决的核心问题

以 AutoGen/CrewAI 这类 Agent 框架为例，它们把“在对话里动态规划与调用工具”作为第一性能力：

场景：用户说“我 8 月 1 号下的单今天还没到，收件地址其实要换，而且我被重复扣费了。”

一个合格的客服 Agent 团队会做什么？

1.**意图识别 + 澄清**

- Planner Agent：拆出多意图（物流异常、改址、计费异常），先问关键澄清（订单号/新地址/扣费凭证）。

2.**跨系统取证**

- OMS/物流工具：查轨迹与 SLA；
- 计费/支付工具：核对重复扣款交易；
- CRM：看是否 VIP、是否有历史补偿记录。

3.**政策推理与合规**

- Policy/Critic Agent：套用“假期延误 + VIP + 改址”的组合条款，评估可给的补偿区间、是否可免费改址、是否触发风控人工复核。

4.**方案生成与协商**

- 提出“改址 + 走加急补发 / 或原包裹拦截 + 退款差额 + 账单冲正”的可行方案，并在对话中按用户反馈实时调整。

5.**执行与闭环**

- 调用工单/票据工具，落账/发券/改单/寄件，写入 CRM 备注；
- 生成总结，告知时限与跟踪号；
- 若任一步失败，自动选择备选策略或升级人工。

这些动作里，很多步骤**无法事先“画”成固定分支，需要在对话上下文里做决策、需要跨工具动态组合、需要“问一句 → 查一下 → 再决定”，**这正是 Agent 的强项。

5.各Agent详细介绍

5.1AutoGPT

简介：AutoGPT是第一个爆火的自主AI Agent框架，提供一系列工具让用户构建和使用自治代理。其功能涵盖代理创建模块“Forge”、性能评测基准agbenchmark、排行榜以及易用的UI和CLI接口。

主要特点：AutoGPT支持“思考-行动-反馈-学习”的循环，让代理不断生成子任务并执行。并且拥有丰富的插件和工具接口，允许代理访问浏览器、文件系统、API等资源，从而完成复杂的链式任务。

典型应用场景：需要让Agent自动拆解目标并执行的，如市场调研、行程规划、代码编写等

优势与不足：

优势	不足
自主性与少人工干预：只需给定最终目标，便能自主规划步骤并连续执行，无需逐步指令指导，从而显著降低人力投入和运营成本	**对话和上下文一致性**：随着任务执行步骤的增多，Agent可能逐渐偏离原定目标，产生与任务无关的输出。模型增加记忆模块可一定程度缓解此问题，但仍不能完全避免上下文丢失和输出偏移现象。
任务分解与多步骤推理：内置了ReAct机制，能够将复杂目标划分为可执行的子任务并逐一完成。并集成了文件操作、网络搜索、代码执行等多种工具，使得AutoGPT在同一框架下即可调用不同能力来解决问题。	**高成本和效率问题**：AutoGPT在执行过程中需要频繁调用大型模型API，每一步决策都可能消耗大量计算资源和费用。此外，AutoGPT采取循环试探的方法执行任务，相较人类直奔主题的处理方式可能显得低效。一些简单任务由AutoGPT执行时迂回冗长，耗时较多。
记忆机制与持续学习：AutoGPT结合了短期与长期记忆模块，能够在对话和操作过程中**保留上下文**、调用先前学到的信息。在连续任务执行中，会将每一步的结果添加进记忆，并据此调整后续行动，从而提高任务完成的连贯性和智能性。这种自我改进能力有助于Agent在长流程任务中表现更佳。	**操作可控性**：由于用户只设定初始目标，过程中Agent的具体操作路径并不透明，可能出现偏离期望的行为。例如，它可能搜索到不相关的信息或尝试执行不恰当的动作而不自知。虽然通常AutoGPT提供了每步执行前让用户确认的选项，但在开放的连续模式下，缺乏监督可能导致错误蔓延。

使用示例：基于AutoGPT让Agent帮我写一篇介绍AutoGPT的文章

1.创建Agent及配置名称、角色以及目标

```
→  Auto-GPT git:(master) ✗ python3 -m autogpt
Weaviate not installed. Skipping import.
pymilvus not installed. Skipping import.
Warning: The file 'AutoGpt.json' does not exist.Local memory would not be saved
to a file.
Welcome to Auto-GPT!  Enter the name of your AI and its role below. Entering not
hing will load defaults.
Name your AI:  For example, 'Entrepreneur-GPT'
AI Name: Smart Writer
Smart Writer here!  I am at your service.
Describe your AI's role:  For example, 'an AI designed to autonomously develop a
nd run businesses with the sole goal of increasing your net worth.'
Smart Writer is: 可以滿足我所有需求的 AI 作家
Enter up to 5 goals for your AI:  For example: Increase net worth, Grow Twitter
Account, Develop and manage multiple businesses autonomously'
Enter nothing to load defaults, enter nothing when finished.
Goal 1: 幫我寫一篇介紹 autogpt 的文章
Goal 2:
Using memory of type:  LocalCache
Using Browser:  chrome
 Thinking...
```

2.Agent 自主思考、规划、执行

```
Goal 1: 幫我寫一篇介紹 autogpt 的文章
Goal 2:
Using memory of type:  LocalCache
Using Browser:  chrome
 THOUGHTS:  I think the best next step would be to start a GPT agent to help me
with writing the article on autogpt.
REASONING:  I believe that starting a GPT agent will help me generate ideas and
content for the article, which will save me time and effort.
PLAN:
-  Start a GPT agent with the task of helping me write an article on autogpt.
-  Provide the agent with a prompt to get started.
-  Use the agent's responses to generate content for the article.
CRITICISM:  I need to make sure that I am not relying too heavily on the GPT age
nt and that I am still using my own judgment to create a high-quality article.
SPEAK:  I will start a GPT agent to help me with writing the article on autogpt.
Attempting to fix JSON by finding outermost brackets
Apparently json was fixed.
NEXT ACTION:  COMMAND = start_agent ARGUMENTS = {'name': 'autogpt_agent', 'task'
: 'Help me write an article on autogpt', 'prompt': 'Can you provide me with some
 information on autogpt and its features?'}
Enter 'y' to authorise command, 'y -N' to run N continuous commands, 'n' to exit
 program, or enter feedback for ...
Input:
```

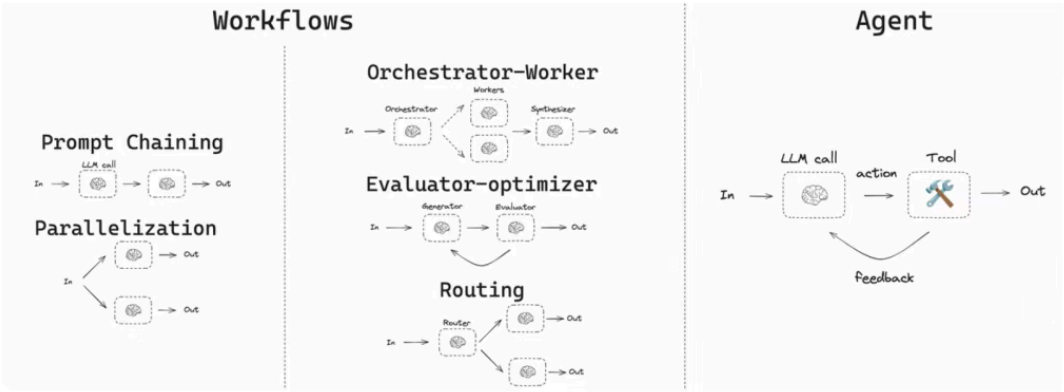
3.最终输出

```
SYSTEM:  Command message_agent returned: Autogpt is a versatile tool that can be
 used in a wide range of industries. Its advanced algorithms and flexibility mak
e it an ideal tool for businesses and individuals who need to create high-qualit
y content quickly and efficiently. Here are some of the key features of autogpt
and its applications in different industries: 1. E-commerce: Autogpt can be used
 to generate product descriptions, reviews, and other content for e-commerce web
sites. This can help businesses to improve their search engine rankings and attr
act more customers. 2. Publishing: Autogpt can be used to generate articles, blo
g posts, and other content for publishing companies. This can help publishers to
 produce more content in less time and reduce their costs. 3. Marketing: Autogpt
 can be used to create engaging content for marketing campaigns. This can help b
usinesses to build brand awareness and increase sales. 4. Education: Autogpt can
 be used to generate educational content, such as textbooks, study guides, and o
nline courses. This can help educators to create high-quality content that is re
levant and engaging for students. 5. Healthcare: Autogpt can be used to generate
 medical content, such as patient education materials and research papers. This
can help healthcare professionals to communicate complex medical information to
patients and other stakeholders. 6. Legal: Autogpt can be used to generate legal
 content, such as contracts, briefs, and legal opinions. This can help lawyers a
nd other legal professionals to produce high-quality content more efficiently. 7
. News: Autogpt can be used to generate news articles and other content for medi
a companies. This can help journalists to produce more content in less time and
keep up with the fast-paced news cycle. In conclusion, autogpt is a powerful too
l that can be used in a wide range of industries. Its advanced algorithms and fl
exibility make it an ideal tool for businesses and individuals who need to creat
e high-quality content quickly and efficiently. Whether you are in e-commerce, p
ublishing, marketing, education, healthcare, legal, or news, autogpt is a tool t
hat you should definitely consider.
```

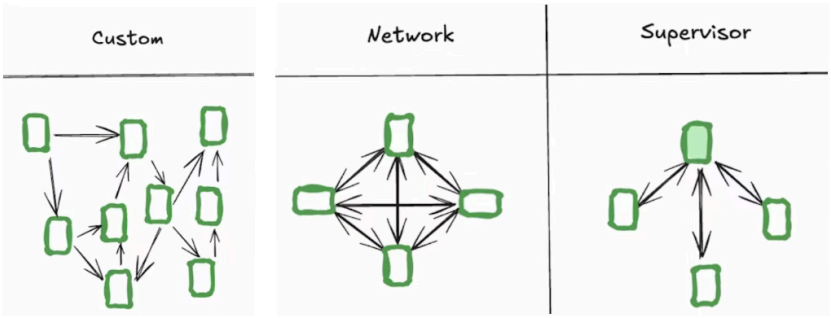
5.2LangGraph

简介：LangGraph 是由 LangChain 团队推出的**有状态、持久运行、多智能体**应用的编排框架。核心将Agent建模成一个**图（Graph）**：每个节点是计算步骤（LLM 调用、工具函数、任意 Python 代码等），边控制流转（含条件与循环），并最终实现既定目标。并且在今年6月提供了**预构建模式**，对常见的多智能体场景提供了抽象封装，开发者只需定义少量参数（如参与的子智能体、主体提示词等）即可快速生成完整的多 Agent 协作系统。

Graph和预构建模式的示意图：



在预定义的链路中加入LLM　　在预定义的链路中由LLM控制流程　　LLM根据环境自行选取工具并执行



主要特点：支持图式编排、可人工干预、可中断/续跑。LangGraph可形成可控的分支/循环流程，可在每个节点中加入人工干预环节，适合需要人工审批/修订的业务场景，并且基于持久化状态可方便中断、续跑、回溯。

典型应用场景：可明确拆解任务步骤的场景，如RAG类、文章生成、日程助手等。

优势与不足：

优势	不足
灵活的多步骤流程控制：LangGraph 最大的优势在于高度灵活的工作流编排能力。通过图结构这种逻辑，使开发者可以针对特定需求定制**非线性**的执行路径，实现从**对话分流**到**复杂工具调用**再到**错误重试**等各种流程。	**自主性有限：LangGraph 强调的是由开发者显式控制**的 Agent 流程（Workflow），这在一定程度上限制了 Agent 的自主性。与 AutoGPT追求高度自我驱动的框架相比，LangGraph 中的智能体基本按照预先设计的图谱执行任务，并不会自行生成新的高层次目标或策略。
LangGraph 引入了共享 State（状态）的概念，在工作流的各节点间持久共享数据。每个节点的输入和输出都可以写入这份共享状态，后续节点能够访问先前步骤的信息。通过这种内存机制，Agent 可拥有短期记忆（对当前对话或当前任务进展的记忆）以及通过外部数据库实现的**长期记忆**。	**预构建模式不成熟：**目前预构建模式**内部交互对用户并不透明**，难以在框架外精确插入自定义逻辑或中间步骤。同时在失败时做特殊处理或并行执行多个任务时，预构建模式缺乏显式机制，**很难实现复杂的流程控制。**预构建模式目前**没有内建的重试、降级或提示机制**，需要开发者在外部捕获并处理，否则可能导致对话中断或不一致。
易调试和高可观察性：由于采用显式的图结构，LangGraph 工作流的执行路径和状态变化透明且可追踪。开发者可以方便地插入日志、检查点，观察数据在各节点的流动，并利用调试工具定位问题。LangGraph 与 LangChain 提供的 LangSmith 等监控/调试工具深度集成，能够对每次 LLM调用、工具使用进行详尽的**跟踪和可视化**，帮助开发者迅速调试复杂链路。	

使用示例：基于LangGraph让Agent帮我写一篇介绍LangGraph的文章

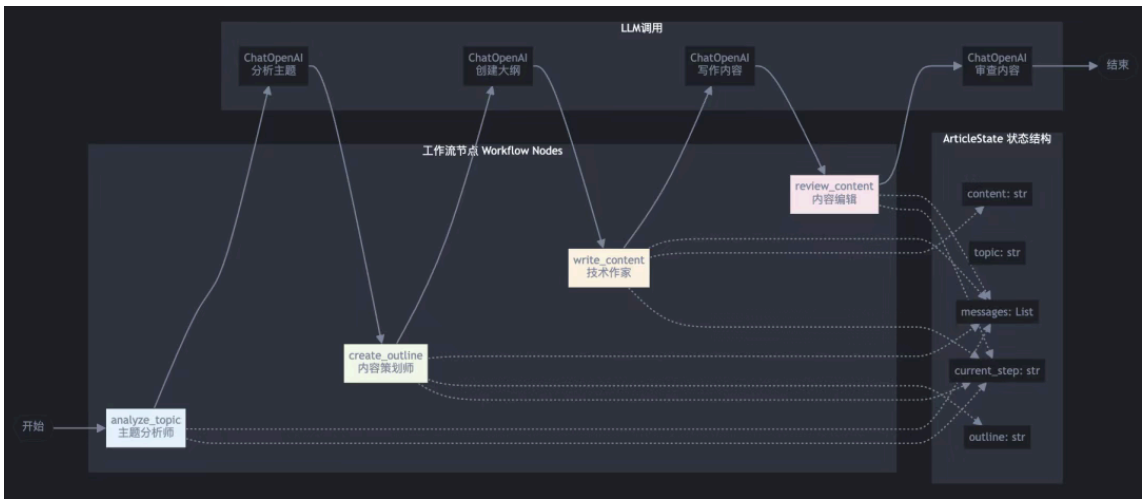
1.构建工作流（Workflow）

```python
def _build_workflow(self) -> StateGraph:
    """构建LangGraph工作流"""
    workflow = StateGraph(ArticleState)

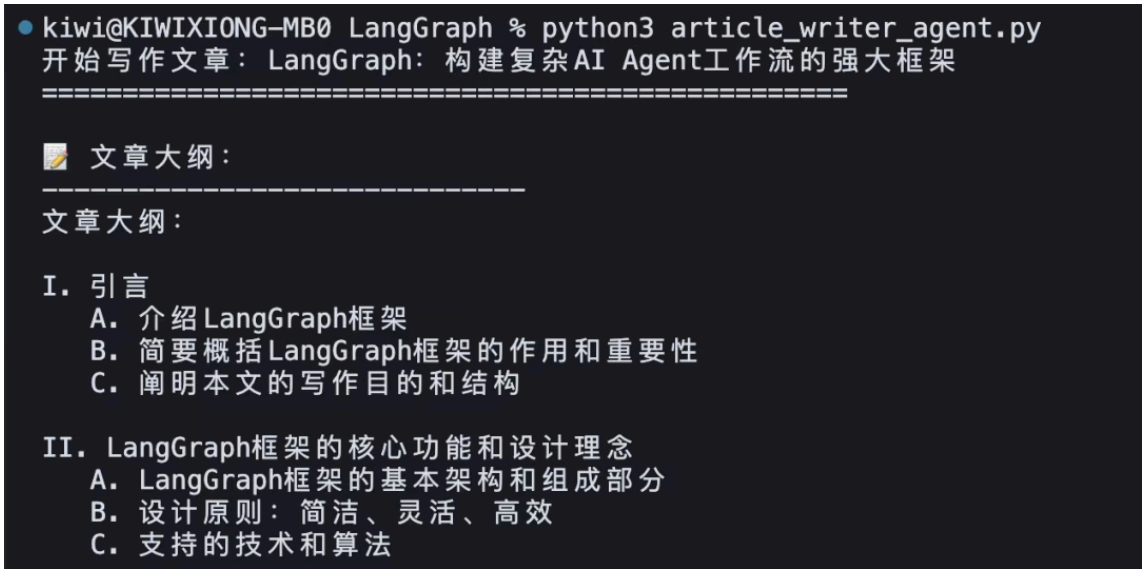
    # 添加节点
    workflow.add_node("analyze_topic", self._analyze_topic)
    workflow.add_node("create_outline", self._create_outline)
    workflow.add_node("write_content", self._write_content)
    workflow.add_node("review_content", self._review_content)

    # 设置边
    workflow.set_entry_point("analyze_topic")
    workflow.add_edge("analyze_topic", "create_outline")
    workflow.add_edge("create_outline", "write_content")
    workflow.add_edge("write_content", "review_content")
    workflow.add_edge("review_content", END)
```

附工作流运行逻辑:



2.最终输出



5.3Dify

简介：Dify（Do It For You）是一个开源的低代码平台，旨在简化大模型（LLM）驱动的AI应用开发与部署。它融合了“后端即服务 (BaaS)”与 LLMOps 概念，提供涵盖模型接入、提示设计、知识库检索、智能代理、数据监控等在内的一站式解决方案。通过直观的可视化界面和预构建组件，开发者和非技术人员都可以快速构建如聊天机器人、内容生成、数据分析等各类生成式AI应用。

主要特点：低代码、可视化工作流构建、检索增强生成（RAG）管道、开放工具市场

典型应用场景：可明确拆解任务步骤的场景，如RAG类、文章生成、日程助手等

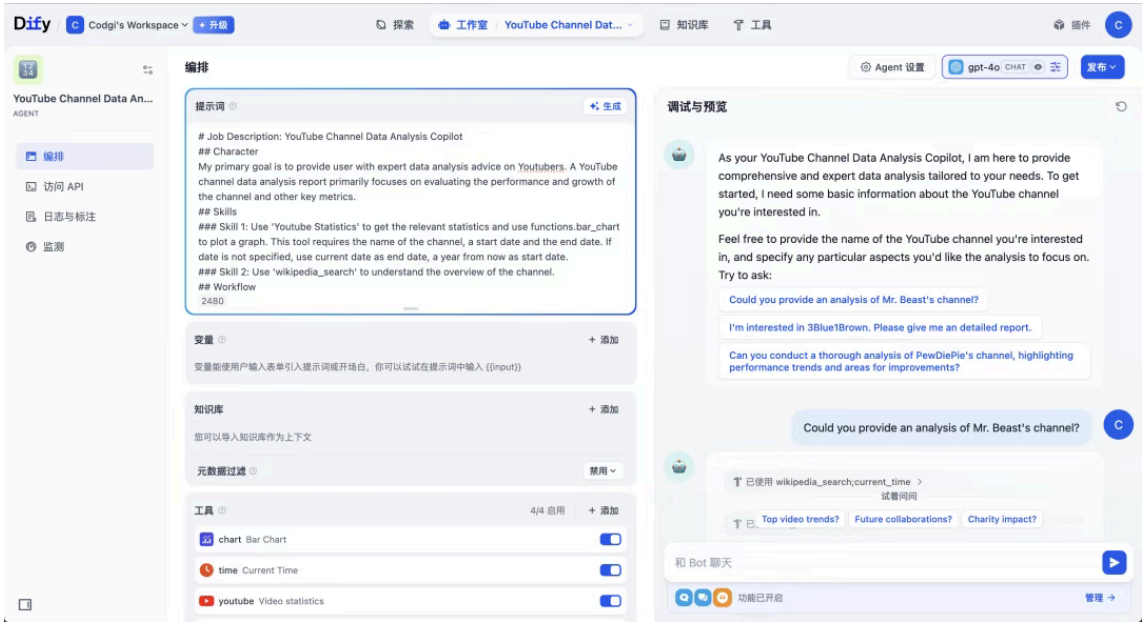
优势	不足
易用性与低门槛： Dify 最大的亮点之一就是**上手非常简单**。其可视化操作界面让用户几乎不需要编码技能就能搭建AI应用。预构建的节点和模板减少了繁琐配置，几小时内即可完成过去需要数周开发的原型。相比要求编程的框架（如 LangChain 等），Dify **降低了 AI 应用开发门槛**，使更多业务人员可以直接参与。	**功能广而不精：** 评价 Dify 它的功能覆盖面很广，但在某些专业领域的深度上可能比不上专门化工具。例如，Dify 内置了知识库RAG功能，但在复杂文档理解、细粒度检索参数方面不及专注RAG的框架（如 RAGFlow 等）
强大的模型与工具集成能力： Dify 生来强调“模型中立”和灵活扩展开箱即支持数十家模型提供商的上百种LLM，涵盖OpenAI、Anthropic、Google、Meta以及各类本地开源模型等。在工具方面，涵盖了常见的网络服务和AI模型，可以借助外部能力完成复杂任务。	**“重量级”工具的取舍：** 如果只是做一个很简单的问答Bot或单一功能，用Dify会感觉“大材小用”，因为它的诸多高级功能用不着，反而增加了系统复杂性。同时，企业如果有很多特殊需求，往往也需要对 Dify 进行二次开发来满足。因此，Dify 最适合的还是**中等复杂度**的场景：太简单的可以直接用现成API，太复杂的可能要深度魔改，在这些边缘情况下，需要权衡使用Dify的性价比。

使用示例：

1.**工作流Workflow类型**



2.**Agent类型（Function Call）**



5.4CrewAI

简介：CrewAI 是一个多智能体（multi-agent）编排框架，其核心理念是让多个具备特定角色的 AI **代理**协同合作（组成“crew”团队）来完成复杂任务。每个代理被赋予特定的角色、目标和背景知识，通过相互分工与配合，自动地进行任务委派和问询，最终以团队形式完成用户交给的工作。

主要特点：多工具及生态集成、支持Workflow和AI Agent两种模式

优势与不足：

优势	不足
工具和生态集成：CrewAI 起初借鉴并**构建在 LangChain 生态**之上，因而天然支持使用 LangChain 提供的大量**工具集合**（如搜索、数据库查询、API 接口等）。同时CrewAI 自身及社区提供了许多内置工具，目前已内置超过 40 种工具接口（包括常用的 LLM、云服务、数据库等）以供代理直接使用。	**特定功能支持有限：** 相较于某些专精的框架，CrewAI 在**特定能力**上可能**不如对手完善**。例如，在"AI编程助手"这一场景中，CrewAI 并没有内置像 AutoGen 那样成熟的**代码执行与自我纠错**循环。如需实现让代理编写并执行代码来完成任务，必须**手动集成额外的工具**（如运行Python代码的工具）。目前 CrewAI 并未直接提供沙箱执行代码的内置模块，这使它在**代码自动生成与执行**的任务上稍显不足。
灵活性与深度定制： 在CrewAI的高层模式下，CrewAI 依然保留了很大的**灵活性**。开发者可以深入定制每个代理的提示（prompt）、工具和内部行为，甚至可以自定义**低层的提示模板和代理行为**。CrewAI 支持同时结合**自主代理（Crews）和精确流程（Flows）两种范式，允许在同一应用中既有自主探索的部分，也有确定顺序的流程，从而无缝融合自治与精确控制。**	

使用示例：研究AIagent领域的最新进展

```yaml
researcher:
  role: >
    {topic} 资深研究员
  goal: >
    {topic} 领域内最新研究成果
  backstory: >
    你是一名资深研究员，擅长{topic}领域内的最新研究成果。
    你能够找到最相关的信息，并将其以清晰明了的方式呈现。

reporting_analyst:
  role: >
    {topic} 报告分析师
  goal: >
    {topic} 领域内最新研究成果
  backstory: >
    你是一名报告分析师，擅长{topic}领域内的最新研究成果。
    你能够将复杂的研究成果转化为清晰明了的报告，使其他人能够轻松理解和采取行动。
```

```yaml
research_task:
  description: >
    全面研究{topic}领域内的最新研究成果
    确保你找到任何有趣和相关信息，当前年份是{current_year}。
  expected_output: >
    10个关于{topic}的最重要的信息列表
  agent: researcher

reporting_task:
  description: >
    审查你得到的上下文，并将每个主题扩展为一个完整的报告部分。
    确保报告详细且包含所有相关信息。
  expected_output: >
    一个完整的报告，包含主要主题，每个主题都有完整的章节信息。
    格式为markdown，不使用'```'
  agent: reporting_analyst
```



5.5AutoGen

简介：AutoGen 是微软开源的一个面向 Agentic AI（代理式人工智能）的编程框架，用于构建 AI 智能体并促进多个智能体协作完成复杂任务。AutoGen 支持事件驱动的分布式架构，具有良好的可扩展性和弹性，可用于搭建可自主行动或在人类监督下运行的多代理 AI 系统。

主要特点：微软开源、原生多Agent支持、灵活对话控制

优势与不足：

优势	不足
原生多代理支持：作为一款专为多智能体协作设计的框架，AutoGen 天生支持多个 Agent 之间的通信与并行工作。它提供了创建和编排多代理对话的高层抽象，使多个 AI 模型可以通过自然语言消息**动态交互**，共同完成任务。	**社区生态尚处于起步阶段**：作为近年才推出的框架（2024 年末发布重构版 v0.4），AutoGen 的生态系统相对其他成熟框架而言仍在成长中。虽然微软提供了详细文档并声称社区支持健全，但由于版本更新较快，**文档偶尔滞后于代码**，出现文档与实际功能不一致的情况。第三方针对 AutoGen 的教程、案例和工具库目前数量有限，大部分资源来自官方团队。这意味着在遇到非常规问题时，开发者能够借鉴的社区经验相对较少，需要更多依赖官方渠道的支持。
灵活的对话流程控制：AutoGen 采用**异步消息驱动**架构，代理之间的通信可以异步进行，不拘泥于固定顺序。这意味着开发者可以实现高度定制的对话流程：代理对话可以根据上下文自由分支、暂停和恢复，甚至在人类干预下重新规划。	
可观察调试支持：框架内置了完备的**可观测性和调试工具**。AutoGen 提供消息跟踪、日志记录以及 OpenTelemetry 集成等功能，方便开发者监控代理间的交互过程，排查问题。此外，AutoGen 允许将代理生成的代码提交到沙盒（如 Docker 容器）安全执行，并支持实时查看代理行为、可视化消息流等。	

Swarm模式下的机票退订助手示例：

```
=== AutoGen Swarm 退票助手 ===
输入 'quit' 或 'exit' 退出程序

---------- TextMessage (user) ----------
我要退订机票
/Users/kiwi/.pyenv/versions/3.10.16/lib/python3.10/site-packages/autogen_agentchat/agents/_assistant_agent.
-2024-08-06 != gpt-4o-2024-11-20. Model mapping in autogen_ext.models.openai may be incorrect. Set the mode
on and suppress this warning.
  model_result = await model_client.create(
---------- ToolCallRequestEvent (travel_agent) ----------
[FunctionCall(id='call_D7qWI3ytrQEoH4QnoI76C2Wo', arguments='{}', name='transfer_to_user')]
---------- ToolCallExecutionEvent (travel_agent) ----------
[FunctionExecutionResult(content='Transferred to user, adopting the role of user immediately.', name='trans
, is_error=False)]
---------- HandoffMessage (travel_agent) ----------
Transferred to user, adopting the role of user immediately.
User: CA1123
---------- HandoffMessage (user) ----------
CA1123
---------- ToolCallRequestEvent (travel_agent) ----------
[FunctionCall(id='call_Lmu3tW8Mewu3WnehwZmCTOR7', arguments='{}', name='transfer_to_flights_refunder')]
---------- ToolCallExecutionEvent (travel_agent) ----------
[FunctionExecutionResult(content='Transferred to flights_refunder, adopting the role of flights_refunder im
l_id='call_Lmu3tW8Mewu3WnehwZmCTOR7', is_error=False)]
---------- HandoffMessage (travel_agent) ----------
Transferred to flights_refunder, adopting the role of flights_refunder immediately.
---------- ToolCallRequestEvent (flights_refunder) ----------
[FunctionCall(id='call_4fHrO1K5uPUL00djWcXe3y7y', arguments='{"flight_id":"CA1123"}', name='refund_flight')]
---------- ToolCallExecutionEvent (flights_refunder) ----------
[FunctionExecutionResult(content='Flight CA1123 refunded', name='refund_flight', call_id='call_4fHrO1K5uPUL
---------- ToolCallSummaryMessage (flights_refunder) ----------
Flight CA1123 refunded
---------- ToolCallRequestEvent (flights_refunder) ----------
[FunctionCall(id='call_KeG4hmEnwsWKYQTPnbij8w0X', arguments='{}', name='transfer_to_travel_agent')]
---------- ToolCallExecutionEvent (flights_refunder) ----------
[FunctionExecutionResult(content='Transferred to travel_agent, adopting the role of travel_agent immediatel
eG4hmEnwsWKYQTPnbij8w0X', is_error=False)]
---------- HandoffMessage (flights_refunder) ----------
Transferred to travel_agent, adopting the role of travel_agent immediately.
---------- TextMessage (travel_agent) ----------
您的机票（CA1123）已经成功退订。如果您需要进一步的帮助，请随时告诉我！祝您有愉快的一天！ TERMINATE
```

6.总结

本篇文章主要介绍了目前 WorkFlow 和 Agent 的区别，以及什么时候应该采用 Agent 框架：当问题复杂、长尾且多变，Agent 才是主力。同时也简要的介绍了目前几类框架如AutoGPT、LangGraph、Dify、CrewAI、AutoGen，希望能在技术路线的选择与框架选型上帮助到各位读者。

腾讯云TDAI（TencentDB AI Service，简称TDAI）团队也在积极探索数据库与 AI 的结合，并正式推出数据库AI服务，为赋予 Agent 长上下文理解与个性化交互能力，腾讯云在数据库AI服务中推出面向 Agent 记忆场景的产品——Agent Memory，负责存储、检索并管理历史交互信息，让AI能够记住并运用这些信息，从而在持续的互动中表现出更强的连贯性、上下文理解力和个性化服务能力。

可以看到，Agent 不只是新的技术名词，更是一种全新的思维方式——让智能系统从“执行命令”走向“理解目标”。未来，在复杂、多变的业务世界中，腾讯云TDAI团队将持续探索从底层存储、索引到记忆调用的完整链路能力，为客户提供 Agent 的基础组件，奠定AI转型的坚实起点。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号



阅读 6万

留言 7

写留言



巴拉迪维 2025年10月22日 4

什么时候也研究并推出一下 Google ADK 吧！



小锋 上海 2025年10月22日 3

人工规划的好处是：确定性、可预测性和可解释性



Billy 北京 2025年10月20日 2

0代码基础，主要用Crewai+LlamaIndex(rag需求实现)+deepseek(编码陪伴)



fish 上海 2025年10月31日 1

@元宝 帮我总结一下



夕夕石 北京 2025年11月9日

不错



吴伦 广东 2025年10月23日

这么好的文为什么没人留言呢~



衿扬 贵州 2025年10月21日

想问下这篇文章是那个团队写的啊？感觉对比得蛮详细的。